

Problem 3: Choosing Architecture for a Scalable Food Delivery Platform

1. Introduction

Food delivery platforms such as **Swiggy** and **Zomato** must serve millions of users while processing thousands of food orders every minute. During peak hours such as lunch and dinner, the system experiences a significant increase in traffic. Therefore, selecting the right software architecture is essential to ensure high performance, reliability, scalability, and availability.

This report recommends the most suitable architecture for a scalable food delivery platform, explains the benefits of Event-Driven Architecture, and identifies the core microservices required for the system.

2. Problem Statement

A startup plans to develop a food delivery application similar to Swiggy or Zomato. The Chief Technology Officer (CTO) must decide whether to use:

- Monolithic Architecture
- Serverless Architecture
- Microservices Architecture

The chosen architecture should support:

- High scalability
- Fast response time
- High availability
- Fault tolerance
- Easy maintenance

3. Recommended Architecture

Microservices Architecture with Event-Driven Architecture (EDA)

The recommended solution is **Microservices Architecture** combined with **Event-Driven Architecture (EDA)**.

In this approach, the application is divided into multiple independent services. Each service performs a specific business function and communicates with other services through events using a message broker such as Apache Kafka or RabbitMQ.

This architecture is widely adopted by modern large-scale platforms because it supports independent deployment, scalability, and fault isolation.

4. Justification for the Recommendation

Reason 1: Independent Scalability

Each microservice can be scaled independently based on demand.

For example:

- During peak hours, the **Order Service** can scale to handle a large number of incoming orders.
- The **Payment Service** can increase processing capacity only when payment requests increase.
- The **Delivery Service** can scale according to the number of available delivery partners.

This approach optimizes resource usage and reduces infrastructure costs.

Reason 2: Better Fault Isolation

A failure in one microservice does not affect the entire application.

For example:

- If the Payment Service experiences an issue, users can still browse restaurants and place orders.
- Delivery tracking and restaurant management continue to function normally.

This improves system reliability and reduces downtime.

Reason 3: Faster Development and Deployment

Each microservice can be developed, tested, and deployed independently by different development teams.

For example:

- Team A works on the Order Service.
- Team B develops the Payment Service.
- Team C maintains the Delivery Service.

This enables faster software releases and simplifies maintenance.

5. Event-Driven Architecture (EDA)

Definition

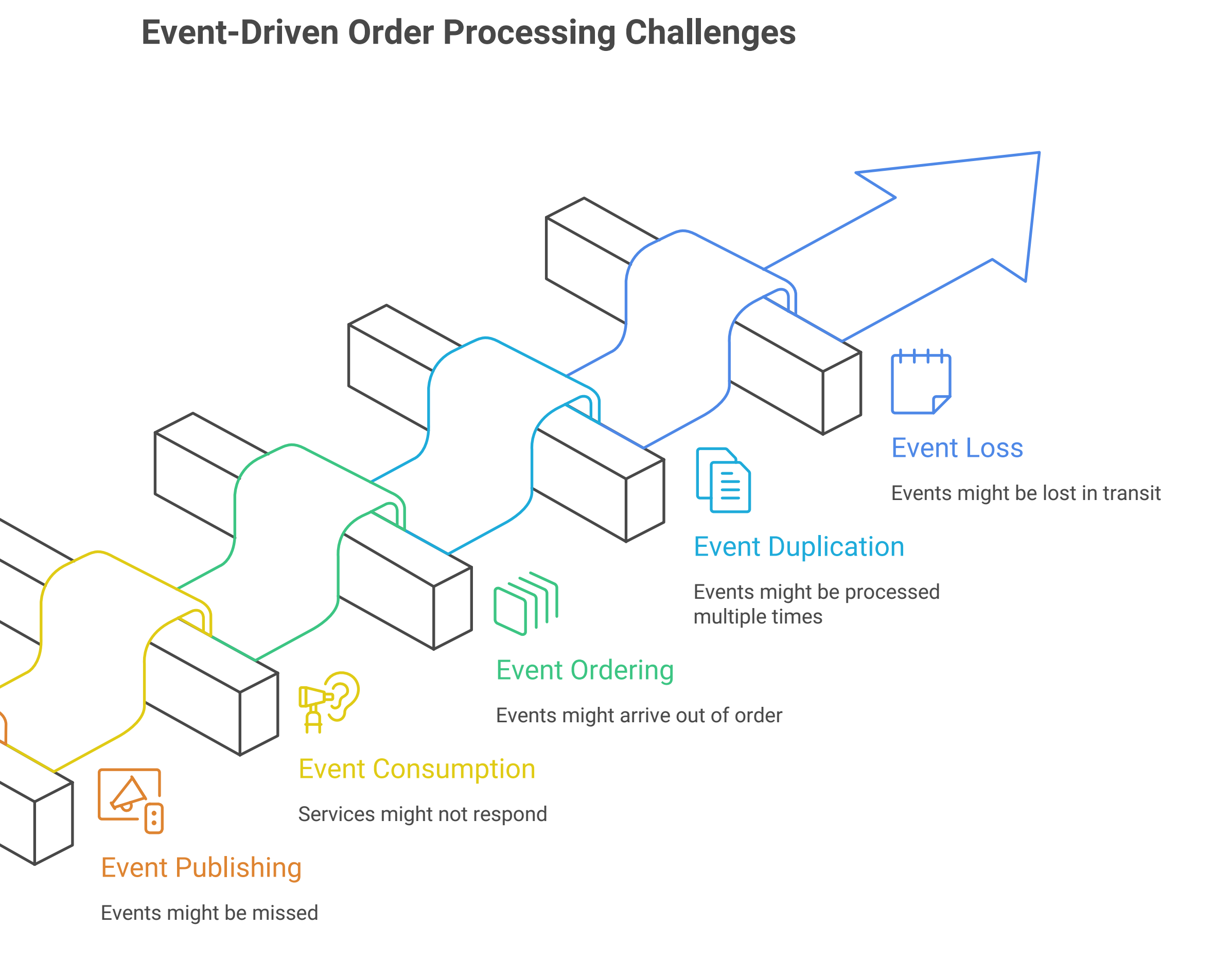
Event-Driven Architecture is a software design pattern in which services communicate by publishing and consuming events rather than making direct requests to each other.

Whenever an important action occurs, an event is generated and shared with other services.

Example events include:

- Order Created
- Payment Successful
- Delivery Assigned
- Order Delivered

6. Working of Event-Driven Architecture



7. Benefits of Event-Driven Architecture

1. Loose Coupling

Services operate independently and communicate through events.

Advantages:

- Easier maintenance
- Independent upgrades
- Better flexibility

2. High Scalability

Events are processed asynchronously, allowing multiple services to handle requests simultaneously.

Benefits:

- Supports thousands of concurrent orders.
- Improves system performance during peak hours.

3. Improved Reliability

Events are stored in a message broker until processed.

If a service becomes temporarily unavailable, the event remains in the queue and is processed once the service is restored.

This prevents data loss and ensures reliable communication.

8. Core Microservices

1. Order Service

Responsibilities

- Create new orders
- Update order status
- Cancel orders
- Maintain order history

2. Payment Service

Responsibilities

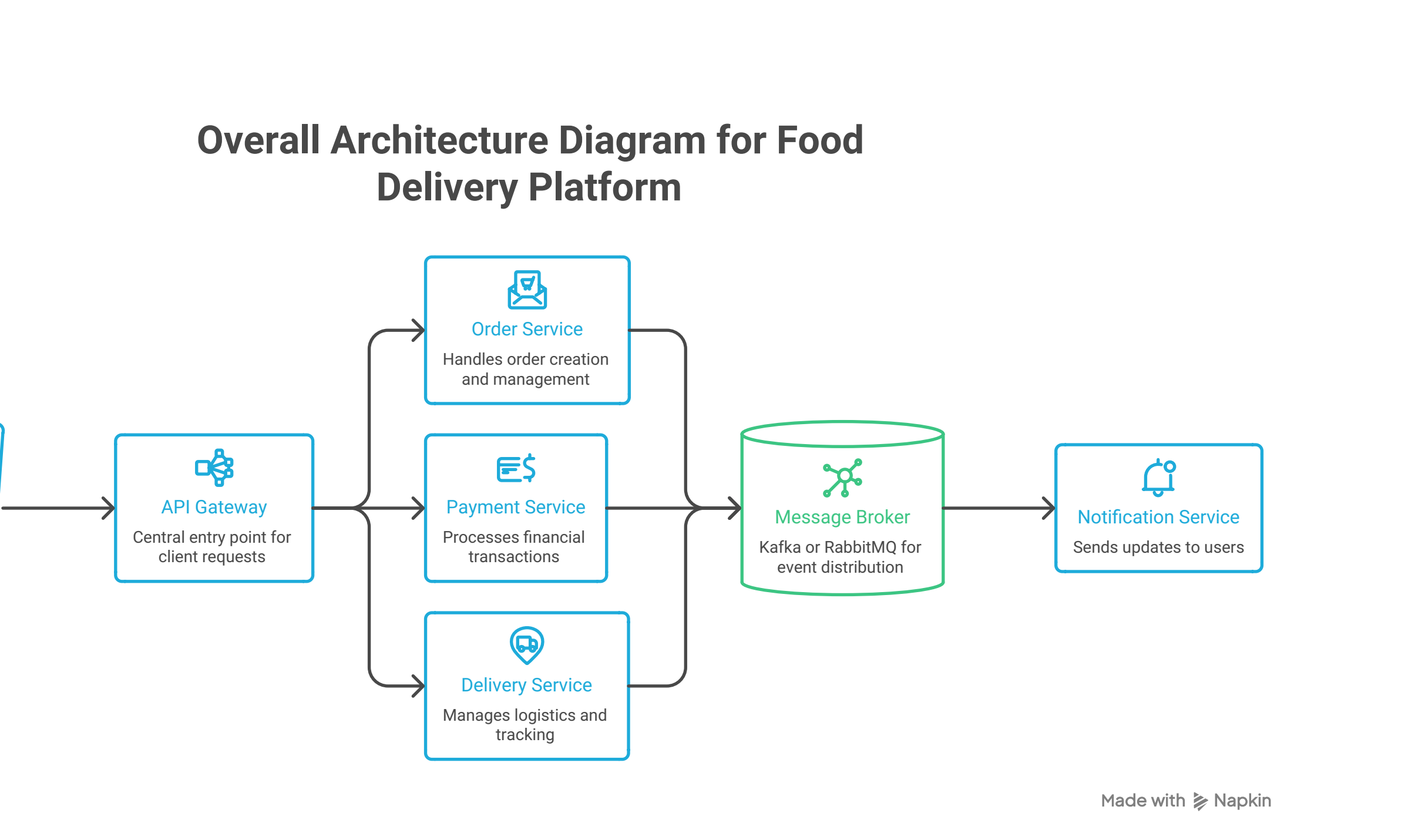
- Process online payments
- Verify payment transactions
- Generate invoices
- Handle refunds

3. Delivery Service

Responsibilities

- Assign delivery partners
- Track delivery locations
- Update delivery status
- Estimate delivery time

9. Overall Architecture Diagram



10. Advantages of the Proposed Architecture

- Supports millions of users efficiently.
- Handles thousands of orders per minute.
- Scales individual services independently.
- Provides high availability and fault tolerance.
- Enables faster software development and deployment.
- Reduces maintenance complexity.
- Improves overall system performance.

11. Conclusion

For a large-scale food delivery platform similar to Swiggy or Zomato, **Microservices Architecture combined with Event-Driven Architecture** is the most appropriate solution. It offers excellent scalability, reliability, maintainability, and fault isolation. By dividing the application into independent services and enabling asynchronous communication through events, the platform can efficiently manage peak-hour traffic while providing a seamless user experience. This architecture also supports continuous development and deployment, making it ideal for modern cloud-based applications.

References

1. Martin Fowler – *Microservices Architecture*
2. Sam Newman – *Building Microservices*
3. Apache Kafka Documentation
4. RabbitMQ Documentation
5. Microsoft Azure Architecture Center – Event-Driven Architecture